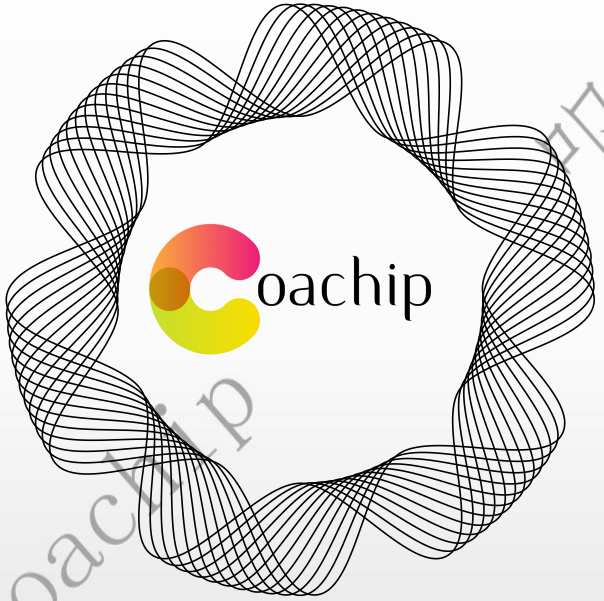


SV面向对象编程

叩特电子Coachip

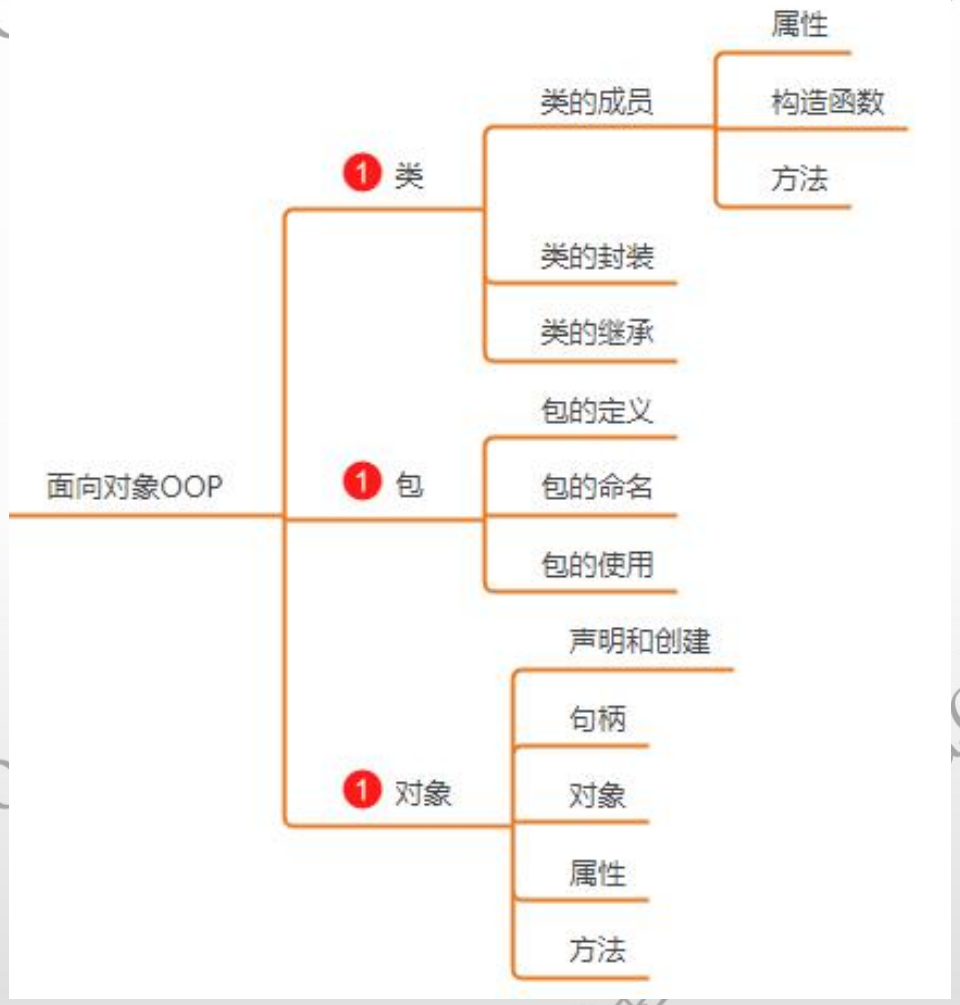
课程内容及目标

课程内容：

- OOP术语
- 类 class
- 包 package
- 对象 object

目标：

- 了解类（class）、对象（object）、句柄（handle）等概念
- 如何使用



叩特电子Coachip

System Verilog OOP 术语

- **Class 类**：包含变量和子程序的基本构建块；在Verilog中module也可以包含变量和方法，只不过它是硬件‘盒子'，而class是软件盒子；
- **Object 对象**：类的一个实例；Verilog中module也可以例化，这是‘硬件'例化，在SV中使用class来例化，这是‘软件'例化；
- **Handle 句柄**：指向对象的指针；在Verilog中，可以通过层次化的索引来找到结构中的设计实例，而在SV的对象索引时，需要通过句柄来索引对象的变量和方法；
- **Property 属性**：存储数据的变量；在Verilog中，它可以是wire或者reg类型；
- **Method 方法**：类中可以使用task或者function来定义方法以便处理自身或者外部传入的数据。在Verilog中可以在module中定义task/function，也可以使用initial/always处理数据。

OOP三大特性五大原则

- 三大特性
  - 封装：封装就是把抽象的数据和对数据进行的操作封装在一起
  - 继承：一个类型获取另外一个类型的属性的方式。分为实现继承和接口继承
  - 多态：同一个行为具有多个不同表现形式或形态的能力，同一消息可以根据发送对象的不同而采用多种不同的行为方式
- 五大原则
  - 开闭原则 Open Close Principle：对扩展开放，对修改关闭；不修改源代码的情况下改变模块的行为
  - 单一职责 Single Responsibility Principle：一个对象应该只包含单一的职责。并将该职责完整的封装到一个类中，高内聚，低耦合；
  - 替换原则Substitution Principle：所有引用基类的地方都能透明的使用其子类
  - 接口隔离原则 Interface segregation Principle：客户端不应该依赖那些它不需要的接口interface
  - 最少知识原则 Least Knowledge Principle：一个软件实体应当尽可能少的与其他实体发生相互作用

叩特电子Coachip

验证为什么需要OOP？

- 那么验证世界中，为什么要有类的存在呢？
  - 激励生成器（stimulus generator）：生成激励内容
  - 驱动器（driver）：将激励以时序形式发送至DUT
  - 监测器（monitor）：监测信号并记录数据
  - 比较器（checker）：比较数据
- 验证环境的**不同组件**其**功能**和所需要处理的**数据**内容是**不相同**的
- 不同环境的**同一类型的组件**其所具备的**功能**和**数据**内容是**相似**的
- 基于以上两点，验证世界的各个组件角色明确、功能分立，使用面向对象编程与验证世界的构建原则十分符合。

叩特电子Coachip

System Verilog类和对象的概述

- **类（class）**是将相同的个体**抽象**出来的描述方式，**对象是实体**。
- 具有相同属性和功能的对象属于同一类，而不同的类之间可能有联系（继承关系）或者没有联系。
- 类的定义核心是属性声明和方法定义，所以类是数据和方法的自洽体（self-compatible），即可以保存数据又可以处理数据。
- 类与struct结构体在数据保存方面的重要区别，结构体只是单纯的数据集合，而类则可以对数据做出符合需要的处理。

即特电子Coachip

一个简单的类

```
class Transaction;
  logic [31:0] addr;
  logic        kind;
  logic [63:0] data;

  function new(logic [31:0] addr);
    this.addr = addr;
    this.kind = 'x;
    this.data = 'x;
  endfunction

  function void display();
    $display("addr=%h, kind=%b, data=%h",
             addr, kind, data);
  endfunction
endclass
```

Property

Constructor

Method

Class

叩特电子Coachip

一个简单的类（续）

```
Transaction tr_gen;
Transaction tr_drv;

initial begin
  tr_gen = new(32'h7788);
  tr_drv = new(32'h8877);

  tr_gen.kind = 1'h1;
  tr_drv.data =  'h0;

  tr_gen.display();
  tr_drv.display();
end
```

Handle

Object

ObjectName.property

ObjectName.Method

叩特电子Coachip

练习：简单类的定义及调用

```
class Transaction;
  bit [31:0] addr, crc, data[8];
  function void display();
    $display("@%0t: TR addr=%h, crc=%h", $time, addr, crc);
    $write("\tdata[0-7]=");
    foreach (data[i]) $write(data[i]);
    $display();
  endfunction
endclass
class PCI_Tran;
  bit [31:0] addr, data; // Use realistic names
  function void display();
    $display("@%0t: PCI: addr=%h, data=%h", $time, addr, data);
  endfunction
endclass
```

叩特电子Coachip

练习：简单类的定义及调用

```
Transaction t;
PCI_Tran pc;
initial begin
  t = new();          // 创建对象
  t.display();        // 调用Transaction::display()
  pc = new();         // 创建对象
  pc.display();       // 调用PCI_Tran::display()
end
```

叩特电子Coachip

Q&A

昂特电子Coachip